

EXPERIMENT-5

Algorithmic Bugging, Optimization & Verification

1. Identified Flaws in Given Slide Algorithm & Flowchart

1. **Loop Termination Logic Error:** The slide states Repeat steps 4.1 and 4.2 until (index $\leq$ num2). In formal algorithmic notation, an *until* condition terminates when the condition evaluates to True. However, the accompanying flowchart incorrectly loops on True and exits on False.
2. **Failure on Negative Second Input:** If num2 is a negative integer (e.g., -5), initializing index = 1 causes the loop check index $\leq$ num2 to evaluate to False on the first iteration. The program exits immediately and prints prod = 0, failing to compute signed multiplication.
3. **Unoptimized Addition Count:** The slide logic adds num1 repeatedly num2 times without checking relative magnitudes. For example, multiplying 3 by 12 results in 12 addition steps instead of optimizing to 3 additions of 12.

2. Corrected Algorithm

1. **Start**
2. **Input:** Read integers num1 and num2.

3. **Zero Validation:** If $\text{num1} == 0$ or $\text{num2} == 0$, set $\text{prod} = 0$, output prod , and stop.

4. **Determine Sign:** Set $\text{is_negative} = (\text{num1} < 0) \text{ XOR } (\text{num2} < 0)$.

5. **Compute Absolute Values:** Set $a = |\text{num1}|$ and $b = |\text{num2}|$.

6. **Minimize Iteration Count:**

- If $a < b$, set $\text{count} = a$ and $\text{addend} = b$.
- Else, set $\text{count} = b$ and $\text{addend} = a$.

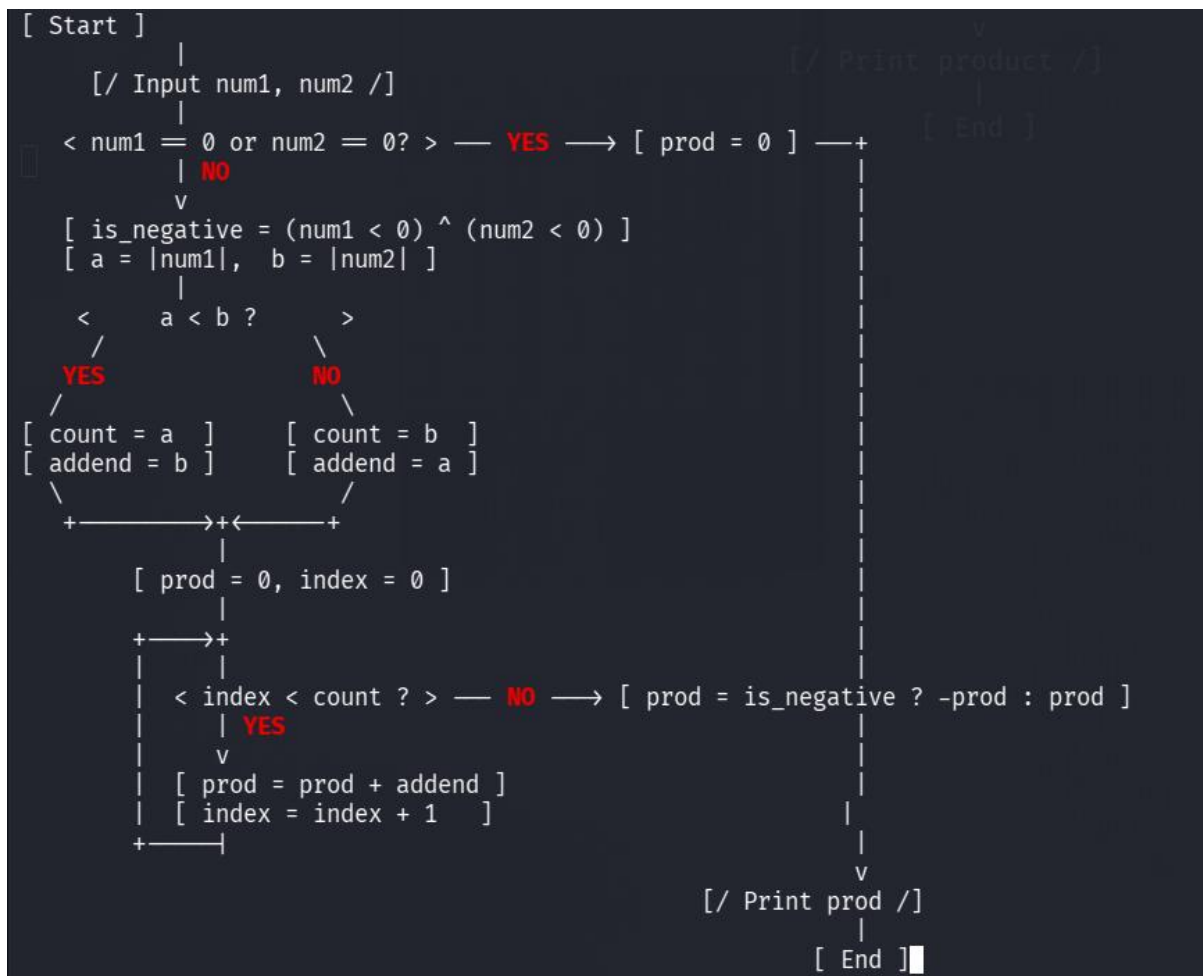
7. **Repeated Addition Loop:**

- Initialize $\text{prod} = 0$, $\text{index} = 0$.
- While $\text{index} < \text{count}$:
 - $\text{prod} = \text{prod} + \text{addend}$
 - $\text{index} = \text{index} + 1$

8. **Apply Sign:** If is_negative is True, set $\text{prod} = -\text{prod}$.

9. **Output:** Print prod .

10. **End**

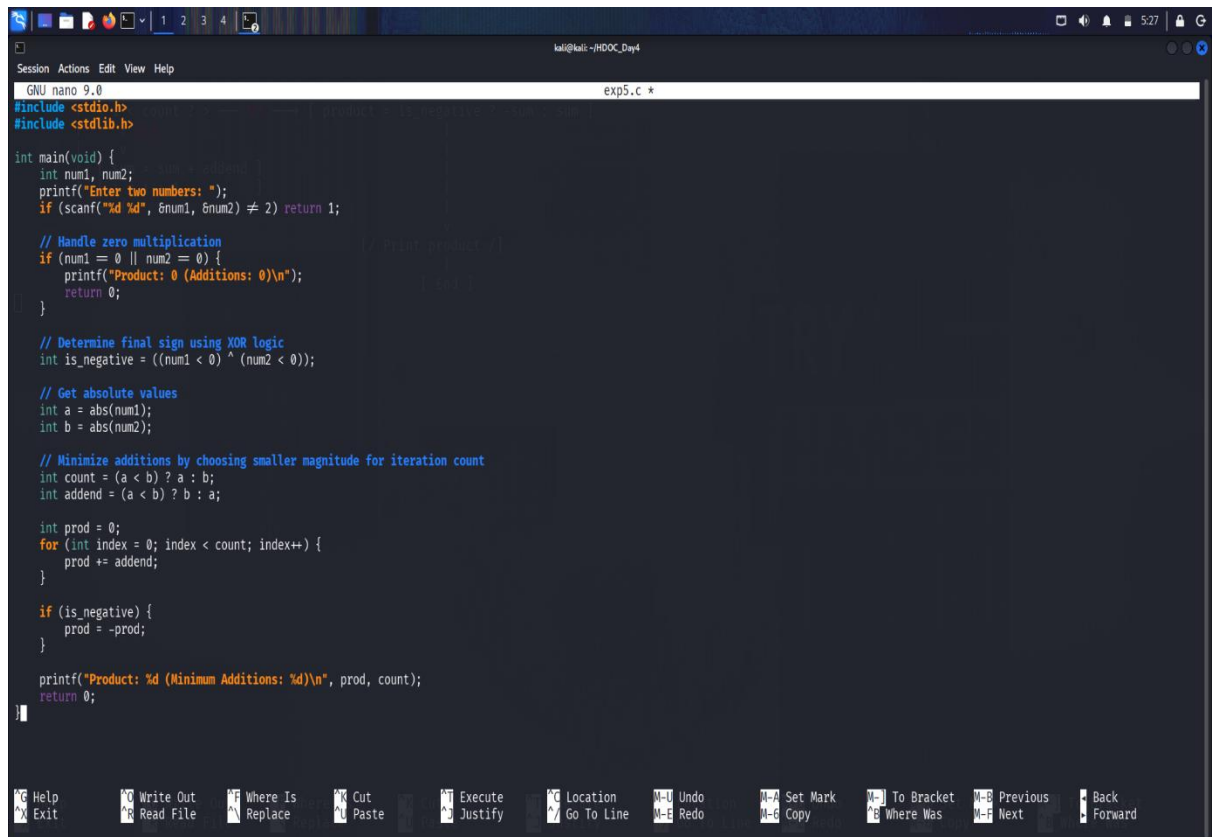


4. Verification Test Cases Table

Test Case	Input (num1, num2)	Expected Output	Minimum Additions
1	12 3	Product: 36	3
2	3 12	Product: 36	3
3	-6 5	Product: -30	5

Test Case	Input (num1, num2)	Expected Output	Minimum Additions
4	6 -5	Product: -30	5
5	-9 -4	Product: 36	4
6	-4 -9	Product: 36	4
7	0 8	Product: 0	0
8	8 0	Product: 0	0
9	0 -7	Product: 0	0
10	-7 0	Product: 0	0
11	0 0	Product: 0	0
12	1 -7	Product: -7	1
13	-1 -9	Product: 9	1

4. C Source Code Implementation



```
GNU nano 9.0 exp5.c *
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int num1, num2;
    printf("Enter two numbers: ");
    if (scanf("%d %d", &num1, &num2) != 2) return 1;

    // Handle zero multiplication
    if (num1 == 0 || num2 == 0) {
        printf("Product: 0 (Additions: 0)\n");
        return 0;
    }

    // Determine final sign using XOR logic
    int is_negative = ((num1 < 0) ^ (num2 < 0));

    // Get absolute values
    int a = abs(num1);
    int b = abs(num2);

    // Minimize additions by choosing smaller magnitude for iteration count
    int count = (a < b) ? a : b;
    int addend = (a < b) ? b : a;

    int prod = 0;
    for (int index = 0; index < count; index++) {
        prod += addend;
    }

    if (is_negative) {
        prod = -prod;
    }

    printf("Product: %d (Minimum Additions: %d)\n", prod, count);
    return 0;
}
```

#include <stdio.h>

#include <stdlib.h>

int main(void) {

int num1, num2;

printf("Enter two numbers: ");

if (scanf("%d %d", &num1, &num2) != 2) return 1;

// Handle zero multiplication

if (num1 == 0 || num2 == 0) {

printf("Product: 0 (Additions: 0)\n");

```
    return 0;  
}
```

```
// Determine final sign using XOR logic
```

```
int is_negative = ((num1 < 0) ^ (num2 < 0));
```

```
// Get absolute values
```

```
int a = abs(num1);
```

```
int b = abs(num2);
```

```
// Swap to ensure minimum additions (smaller number  
becomes loop counter)
```

```
int count = (a < b) ? a : b;
```

```
int addend = (a < b) ? b : a;
```

```
int prod = 0;
```

```
for (int index = 0; index < count; index++) {
```

```
    prod += addend;
```

```
}
```

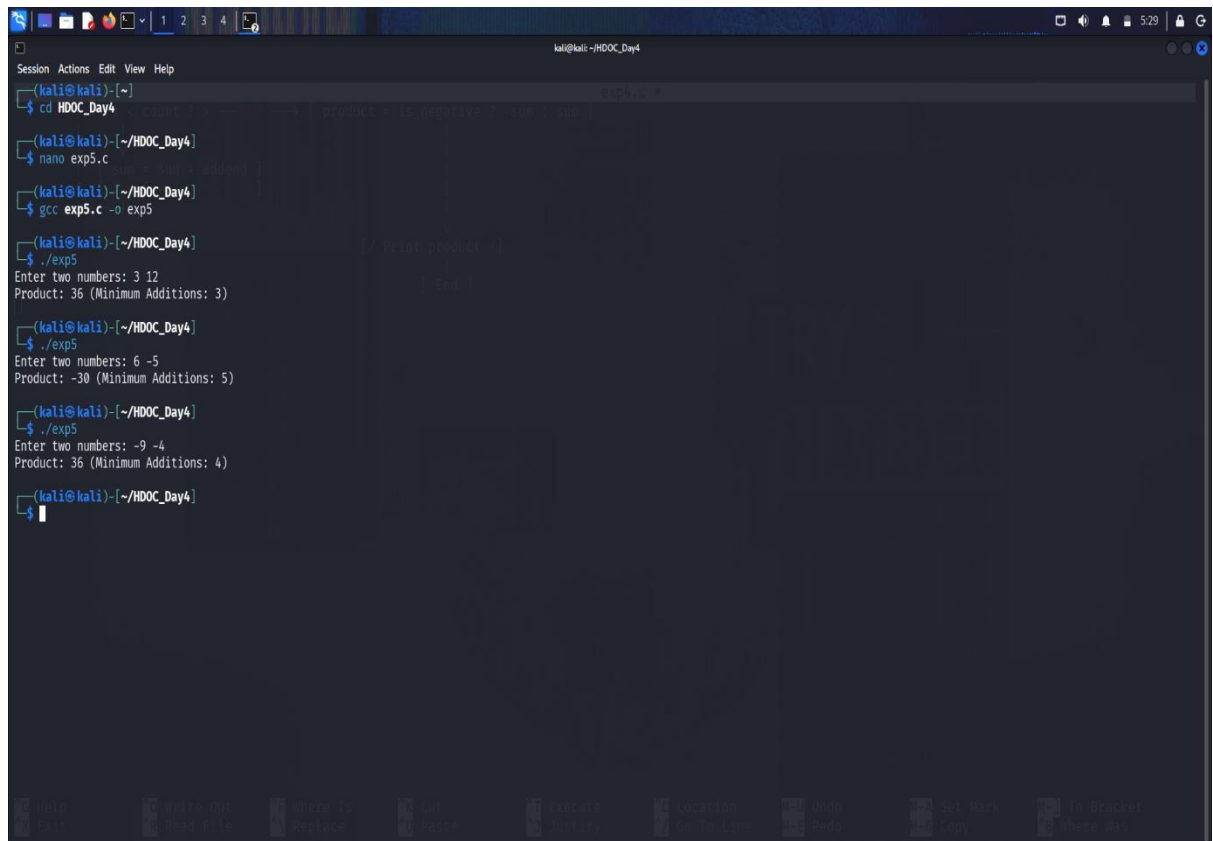
```
if (is_negative) {
```

```
    prod = -prod;
}

printf("Product: %d (Minimum Additions: %d)\n", prod,
count);

return 0;
}
```

5. Execution & Terminal Proof



```
kali@kali: ~/HDOC_Day4
$ cd HDOC_Day4
$ nano exp5.c
$ gcc exp5.c -o exp5
$ ./exp5
Enter two numbers: 3 12
Product: 36 (Minimum Additions: 3)

$ ./exp5
Enter two numbers: 6 -5
Product: -30 (Minimum Additions: 5)

$ ./exp5
Enter two numbers: -9 -4
Product: 36 (Minimum Additions: 4)

$
```